



第九章 安全多方计算

目 录

『01』

知识点一：姚氏百万富翁问题

『02』

知识点二：混淆电路构造思想

『03』

知识点三：混淆电路求解示例

『04』

知识点四：姚氏乱码电路协议

1. 问题描述

- 姚期智 (Andrew Chi-Chih Yao) 是中国第一个也是目前为止唯一的图灵奖获得者。他为现代密码学打开了一道新的大门。其中一项重要贡献，就是安全多方计算。
- 他提出了著名的姚氏百万富翁问题：

假如有两个百万富翁 Alice 和 Bob 各有钱 i 和 j ，他们想知道谁的钱多，但是又不想把自己的钱数量告诉对方，怎么样才能比大小呢？

2. 姚先生的求解

- (1) 先把问题简化, 假如 i 和 j 是1到10之间的数。
- (2) 首先Bob挑选一个大整数 x , 然后用Alice的公钥加密得到 $k = \text{Enc}(x)$ 。

然后把数 $k-j+1$ 发给Alice。

- (3) Alice拿到这个数之后, 计算以下这些数:

$$\text{Dec}(k-j+1), \text{Dec}(k-j+2), \text{Dec}(k-j+3), \dots, \text{Dec}(k-j+10)$$

然后除以一个素数 p 取余得到:

$$z_1 = \text{Dec}(k-j+1)(\text{mod } p), z_2, z_3, \dots, z_{10}$$

2. 姚先生的求解

因为Alice的钱是 i ，那么Alice 做一下这个事情：

$$z_1, z_2, z_3, \dots, z_{i-1}, z_i = z_i + 1, z_{i+1} = z_{i+1} + 1, \dots$$

也就是将第 i 及后面的数都加1，然后把这一串数字发给Bob。

□ (4) Bob只需要看第 j 个数字：

- 如果等于 $x \pmod{p}$ 就说明 $i > j$ ；
- 否则，说明 $i \leq j$ 。

然后，Bob把结果返给Alice。

第 j 个数字，即 $k-j+j = k = \text{Enc}(x)$

如果相等，说明第 j 个数字没有改动过，处于左侧

目 录

『01』

知识点一：姚氏百万富翁问题

『02』

知识点二：混淆电路构造思想

『03』

知识点三：混淆电路求解示例

『04』

知识点四：姚氏乱码电路协议

1. 基本思想

- Yao的上述百万富翁求解的协议的交点仅限于比较大小，而其他的运算还没有支持。当然后来一个在此基础上一个更加伟大的密码学算法被发明出来了，那就姚氏乱码电路（Garbled Circuit, GC），是最著名、最广为人知的MPC协议。
- 混淆电路的思想很简单，它源于一个事实：**可以通过设计电路来对目标问题进行求解**。电路可以通过与或非门实现任意一个函数，而多方计算的目标就是保护各方输入信息的情况下进行目标函数的计算。

1. 基本思想

- 对参与运算的双方，从参与者的视角可以分为**电路产生者(circuit generator)**与**电路执行者(circuit evaluator)**。混淆电路有两个阶段：
 - **电路产生阶段：**电路产生者将待计算的函数转化为电路。参与运算的双方先就需要安全计算的目的依靠专有编程语言(DSL)或相关编程语言扩展等进行编程，然后针对实现计算的程序进行编译，生成电路文件；
 - **电路执行阶段：**电路执行者安全地计算电路。利用不经意传输 (Oblivious Transfer, OT)、加密等密码学原语等执行电路，在不泄露电路的输入和中间结果的前提下，完成电路的计算。

2. 基于查找表的构造思想

回顾一下，为了要求解函数 $F(x, y)$ 的值，**参与方 P_1 持有 $x \in X$** ，**参与方 P_2 持有 $y \in Y$** 。这里的 X 和 Y 分别为 P_1 和 P_2 的输入域。

□ 第一、将函数表示为查找表

我们首先考虑一个输入域很小的函数 F 。由于 F 的输入域很小，我们可以很快地**枚举出所有可能的输入对 (x, y)** 。

可以把函数 F 表示为一个包含 $|X| \cdot |Y|$ 行的查找表 T ，每行的条目为 $T_{x,y} = \langle F(x, y) \rangle$ ，即**该表只有1列，共有 $|X| \cdot |Y|$ 行**。

只要能定位 $T_{x,y}$ ，就可以获得 $F(x, y)$ 的输出。

1	1	$f(1, 1)$
1	2	$f(1, 2)$
1	3	$f(1, 3)$
1	4	$f(1, 4)$
2	1	$f(2, 1)$
2	2	$f(2, 2)$
2	3	$f(2, 3)$
2	4	$f(2, 4)$
3	1	$f(3, 1)$
3	2	$f(3, 2)$
3	3	$f(3, 3)$
3	4	$f(3, 4)$
4	1	$f(4, 1)$
4	2	$f(4, 2)$
4	3	$f(4, 3)$
4	4	$f(4, 4)$

2. 基于查找表的构造思想

可以按照下述方式生成查找表：

- ① P_1 通过为每一个可能的输入 x 和 y 随机指定一个强密钥来加密 T 。也就是说，对于每一个 $x \in X$ 和每一个 $y \in Y$ ， P_1 将选择 $k_x \in_R \{0,1\}^k$ 和 $k_y \in_R \{0,1\}^k$ 。

A_1	B_1	$f(1, 1)$
A_1	B_2	$f(1, 2)$
A_1	B_3	$f(1, 3)$
A_1	B_4	$f(1, 4)$
A_2	B_1	$f(2, 1)$
A_2	B_2	$f(2, 2)$
A_2	B_3	$f(2, 3)$
A_2	B_4	$f(2, 4)$
A_3	B_1	$f(3, 1)$
A_3	B_2	$f(3, 2)$
A_3	B_3	$f(3, 3)$
A_3	B_4	$f(3, 4)$
A_4	B_1	$f(4, 1)$
A_4	B_2	$f(4, 2)$
A_4	B_3	$f(4, 3)$
A_4	B_4	$f(4, 4)$

2. 基于查找表的构造思想

可以按照下述方式生成查找表：

- ① P_1 通过为每一个可能的输入 x 和 y 随机指定一个强密钥来加密 T 。也就是说，对于每一个 $x \in X$ 和每一个 $y \in Y$ ， P_1 将选择 $k_x \in_R \{0,1\}^K$ 和 $k_y \in_R \{0,1\}^K$ 。
- ② 随后， P_1 同时使用两个密钥 k_x, k_y 加密 T 中相应的数据项 $T_{x,y}$ 。

$\mathbb{E}_{A_1, B_1}(f(1, 1))$
$\mathbb{E}_{A_1, B_2}(f(1, 2))$
$\mathbb{E}_{A_1, B_3}(f(1, 3))$
$\mathbb{E}_{A_1, B_4}(f(1, 4))$
$\mathbb{E}_{A_2, B_1}(f(2, 1))$
$\mathbb{E}_{A_2, B_2}(f(2, 2))$
$\mathbb{E}_{A_2, B_3}(f(2, 3))$
$\mathbb{E}_{A_2, B_4}(f(2, 4))$
$\mathbb{E}_{A_3, B_1}(f(3, 1))$
$\mathbb{E}_{A_3, B_2}(f(3, 2))$
$\mathbb{E}_{A_3, B_3}(f(3, 3))$
$\mathbb{E}_{A_3, B_4}(f(3, 4))$
$\mathbb{E}_{A_4, B_1}(f(4, 1))$
$\mathbb{E}_{A_4, B_2}(f(4, 2))$
$\mathbb{E}_{A_4, B_3}(f(4, 3))$
$\mathbb{E}_{A_4, B_4}(f(4, 4))$

2. 基于查找表的构造思想

可以按照下述方式生成查找表：

- ① P_1 通过为每一个可能的输入 x 和 y 随机指定一个强密钥来加密 T 。也就是说，对于每一个 $x \in X$ 和每一个 $y \in Y$ ， P_1 将选择 $k_x \in_R \{0,1\}^k$ 和 $k_y \in_R \{0,1\}^k$ 。
- ② 随后， P_1 同时使用两个密钥 k_x, k_y 加密 T 中相应的数据项 $T_{x,y}$ 。
- ③ 最后，将加密且经过随机置换的查找表 $\langle Enc_{x,y}(T_{x,y}) \rangle$ 发送给 P_2 。

$E_{A_3, B_4}(f(3, 4))$
$E_{A_4, B_3}(f(4, 3))$
$E_{A_3, B_3}(f(3, 3))$
$E_{A_2, B_3}(f(2, 3))$
$E_{A_4, B_2}(f(4, 2))$
$E_{A_2, B_4}(f(2, 4))$
$E_{A_4, B_4}(f(4, 4))$
$E_{A_1, B_4}(f(1, 4))$
$E_{A_2, B_2}(f(2, 2))$
$E_{A_1, B_2}(f(1, 2))$
$E_{A_2, B_1}(f(2, 1))$
$E_{A_1, B_3}(f(1, 3))$
$E_{A_4, B_1}(f(4, 1))$
$E_{A_3, B_1}(f(3, 1))$
$E_{A_1, B_1}(f(1, 1))$
$E_{A_3, B_2}(f(3, 2))$

2. 基于查找表的构造思想

□ 第二、利用茫然传输隐藏行内访问的元素

我们现在的任务是让 P_2 （只能）解密与参与方输入相关联的数据项 $T_{x,y}$ 。具体实现方

式是让 P_1 向 P_2 （茫然的）发送密钥 k_x 和 k_y ：

- P_1 已知自己的输入 x ，因此 P_1 只需要将密钥 k_x 直接发送给 P_2 即可（因为只是发送了一次性选的密钥，这个过程不会泄露 x ）。
- P_2 已知自己的输入 y ，它可以通过 $|M|$ 选1-OT协议从 P_1 获得 k_y 。

一旦收到 k_x 和 k_y ， P_2 就可以使用这些密钥解密 $T_{x,y}$ ，得到输出 $F(x,y)$ 。

2. 基于查找表的构造思想

- 最重要的是, P_2 在此过程中无法获得任何其他信息。这是因为 P_2 只拥有一对密钥, 只能打开 (解密) 查找表中的一个数据项。
- 需要特别强调的是, 单独使用 k_x 或 k_y 都不允许部分解密密文, 甚至不能单独使用 k_x 或 k_y 判断某个密文是否是用 k_x 或 k_y 加密得到的。

$\mathbb{E}_{A_3, B_4}(f(3, 4))$
$\mathbb{E}_{A_4, B_3}(f(4, 3))$
$\mathbb{E}_{A_3, B_3}(f(3, 3))$
$\mathbb{E}_{A_2, B_3}(f(2, 3))$
$\mathbb{E}_{A_4, B_2}(f(4, 2))$
$\mathbb{E}_{A_2, B_4}(f(2, 4))$
$\mathbb{E}_{A_4, B_4}(f(4, 4))$
$\mathbb{E}_{A_1, B_4}(f(1, 4))$
$\mathbb{E}_{A_2, B_2}(f(2, 2))$
$\mathbb{E}_{A_1, B_2}(f(1, 2))$
$\mathbb{E}_{A_2, B_1}(f(2, 1))$
$\mathbb{E}_{A_1, B_3}(f(1, 3))$
$\mathbb{E}_{A_4, B_1}(f(4, 1))$
$\mathbb{E}_{A_3, B_1}(f(3, 1))$
$\mathbb{E}_{A_1, B_1}(f(1, 1))$
$\mathbb{E}_{A_3, B_2}(f(3, 2))$

2. 基于查找表的构造思想

□ 第三、安全求解函数 $F(x, y)$

由于置换表 T 是随机置换后发给 P_2 , P_2 仅仅得到了解密密钥,
它如何知道 (x, y) 对应的数据项呢?

解决这个问题最简单的方法是在 T 的加密条目中编码一些附加信息。
例如, P_1 可以在 T 的每一行字符串的末尾附加 σ 个 0。如果解密了错误的行, 则解密结果的末尾仅有很低的概率 ($p = \frac{1}{2^\sigma}$) 包含 σ 个 0, 这样 P_2 就可以知道解密结果是错误的。

找一位同学，再重复一下基于查找表的构造思想。

3. 优化方法

□ 利用标识置换实现行的定位

- 1990年，Beaver等人提出了标识置换（Point-and-Permute）技术，可以确定特定输入对应的密文数据项在置乱后的查找表中的位置。
- 此方法的基本思想是**将密钥的一部分（即第一个密钥的后 $\lceil \log |X| \rceil$ 比特和第二个密钥的后 $\lceil \log |Y| \rceil$ 比特）作为查找表 T 的置换标识，标识密钥将加密哪行（或者哪个位置）密文，并根据置换标识对加密后的查找表进行置换。**
- 在具体协议中，可以设计为：
 - **先确定查找表的置换标识，**
 - **然后根据这个置换标识完成置换。**

同样，在求解的时候，
得到密钥就可以知道密文对应的位置了

3. 优化方法

□ 管理查找表的大小

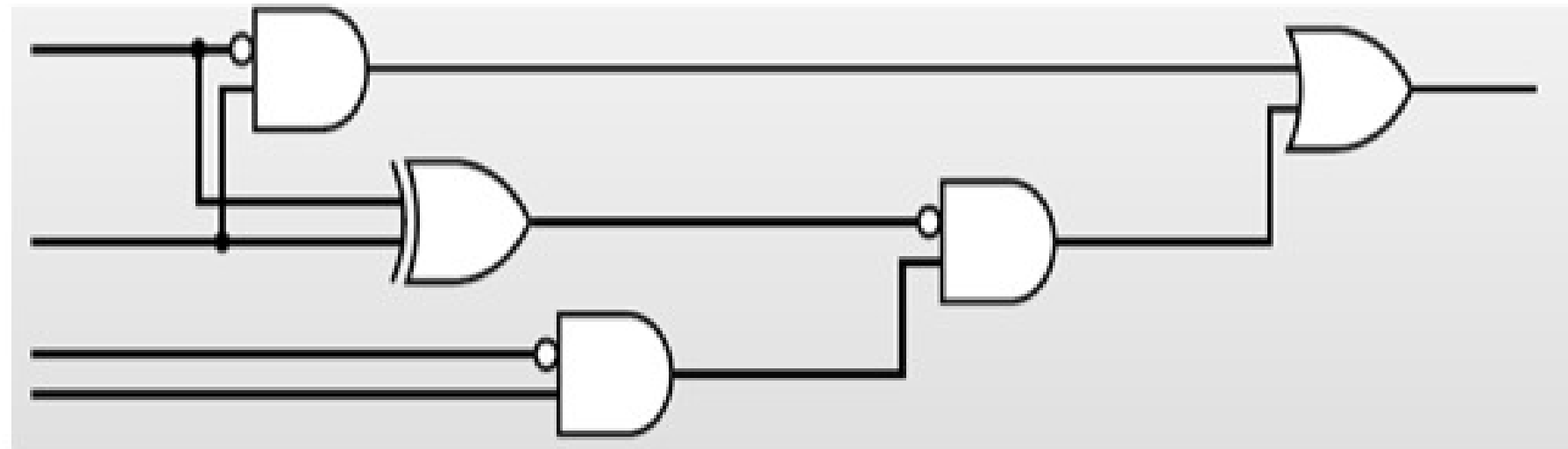
- 显然，上述方案的效率较低，因为查找表的大小与 F 的定义域大小呈线性关系。但是，对于布尔电路门这样的小型函数，其定义域的大小仅为4，用查找表表示此类小型函数是比较高效的。
- 因此，降低查找表的大小的思路是：

将 F 表示为布尔电路 C ，并用定义域大小为4的查找表
求解每一个门的输出。

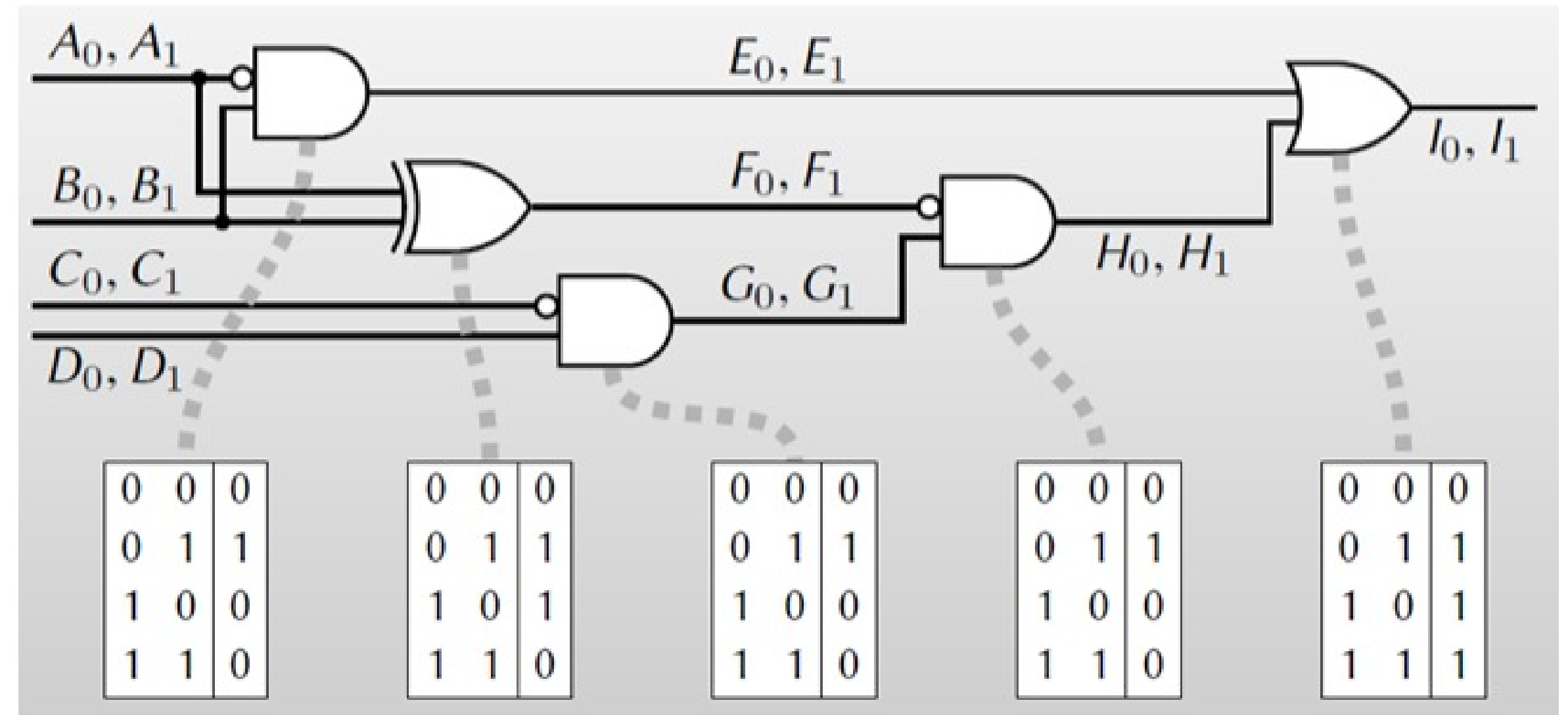
对于布尔电路而言，电路实现与或非即可实现完备，可以模拟任意的函数。

3. 优化方法

□ 管理查找表的大小



用布尔电路表示F



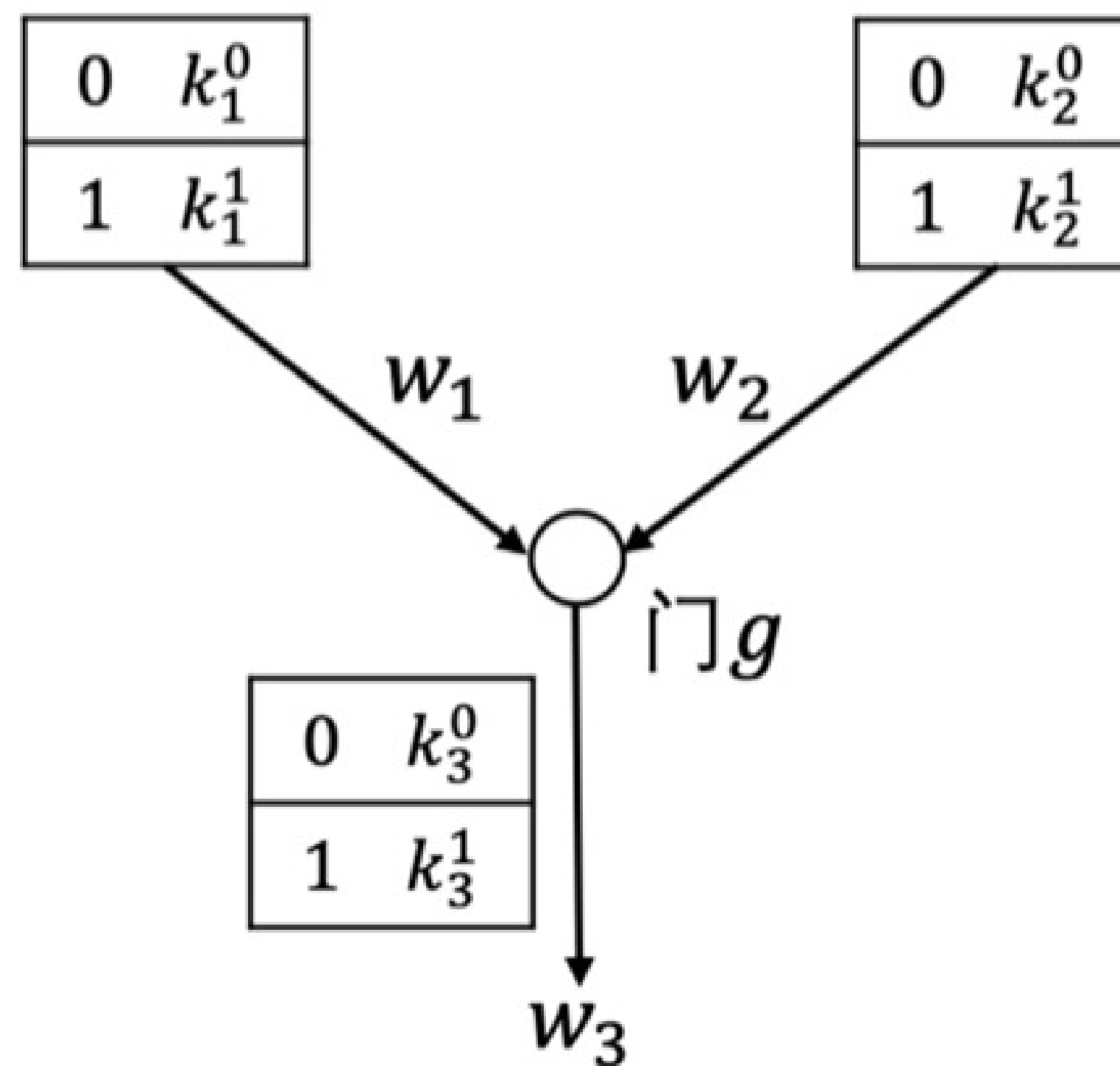
为每个门构建查找表

有关姚氏混淆电路，说法错误的是

- A 借鉴了查找表的思想来实现安全计算
- B 使用茫然传输来安全的获得求值方拥有的输入所对应的密钥
- C 可以在加密条目中加入一些附加信息，使得可以鉴别解密的数据，但是这个方式效率比较低
- D 标识置换因为将位置附加到密钥后面，因此会泄露密文的位置，使得混淆电路不安全

4. 混淆电路生成

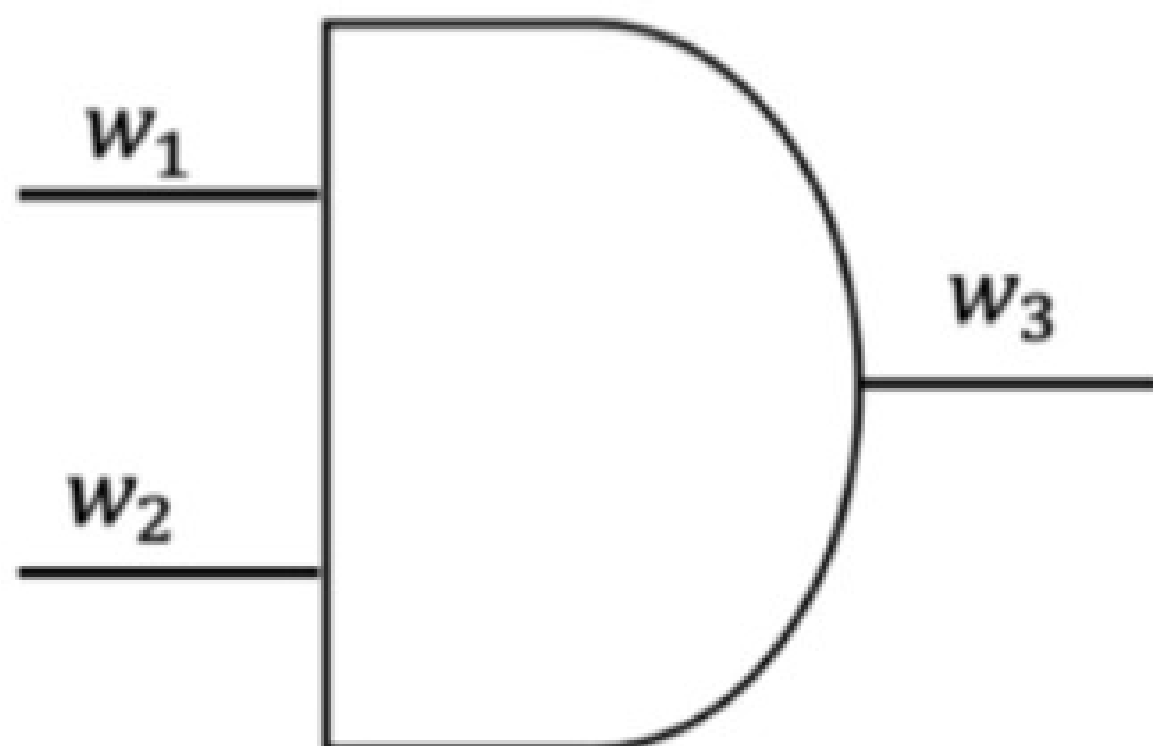
- 布尔电路的基础结构如右图所示，门 g 可以是与门、或门等，它接收两个输入，输出一个结果。



4. 混淆电路生成

□ 生成混淆电路

首先以与门为例，一个常见的与门及其真值表（Truth Table）如下图所示，将该与门的输入线记为 w_1 , w_2 ，输出线记为 w_3 。



w_1	w_2	w_3
0	0	0
0	1	0
1	0	0
1	1	1

4. 混淆电路生成

w_1	w_2	w_3
0	0	0
0	1	0
1	0	0
1	1	1



w_1	w_2	w_3
k_1^0	k_2^0	$En_{k_1^0, k_2^0}(k_3^0)$
k_1^0	k_2^1	$En_{k_1^0, k_2^1}(k_3^0)$
k_1^1	k_2^0	$En_{k_1^1, k_2^0}(k_3^0)$
k_1^1	k_2^1	$En_{k_1^1, k_2^1}(k_3^1)$

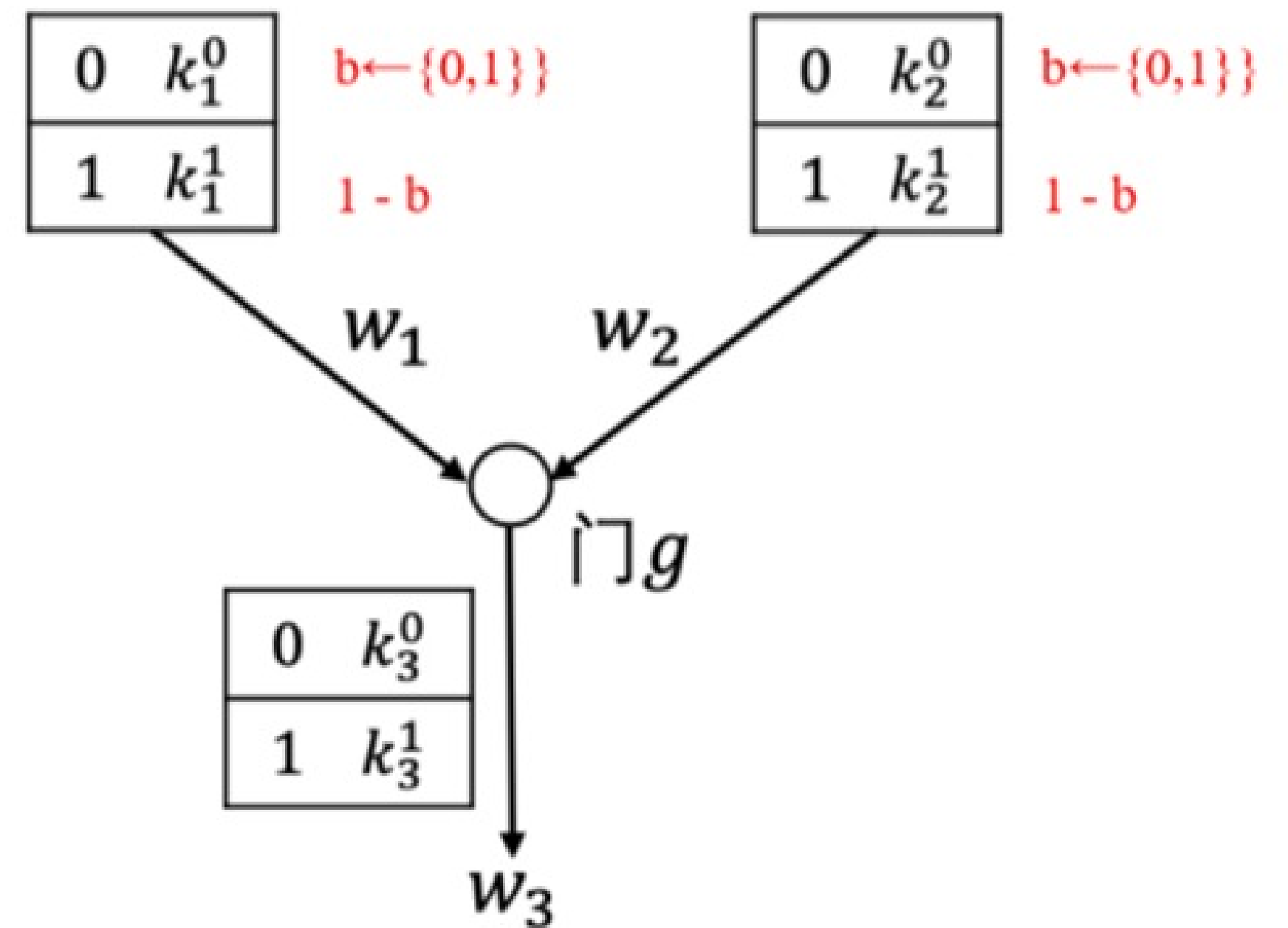
- 随机生成6个密钥 $k_1^0, k_1^1, k_2^0, k_2^1, k_3^0, k_3^1$ ，分别表示 w_1, w_2, w_3 这三条线为0和1时的两种情况。如 k_1^0, k_1^1 分别代表 w_1 为0和 w_1 为1， k_3^0, k_3^1 分别代表 w_3 为0和 w_3 为1。我们称这些密钥为**导线标签 (Wire Label)**，称导线的明文值为**导线值 (Wire Value)**。
- 接着该门利用对称加密算法 $En()$ 生成4个密文 $c_{0,0}, c_{0,1}, c_{1,0}, c_{1,1}$ 。 $En_{a,b}(c)$ 表示用 a, b 作为加密密钥、使用加密算法 $En()$ 来加密 c ，即 $c_{a,b} = En_{a,b}(c) = En_a(En_b(c))$ 。

4. 混淆电路生成

随机置换查找表。 $T_G = \{c_{0,0}, c_{0,1}, c_{1,0}, c_{1,1}\}$ 就是 P_1 构建的**初始查找表**，查找表中的每一行条目都是门输出值所对应导线标签的密文。考虑安全性，需要对查找表进行随机置乱。

在查找表只包含4行密文的情况下，**标识置换**技术非常简单和高效：

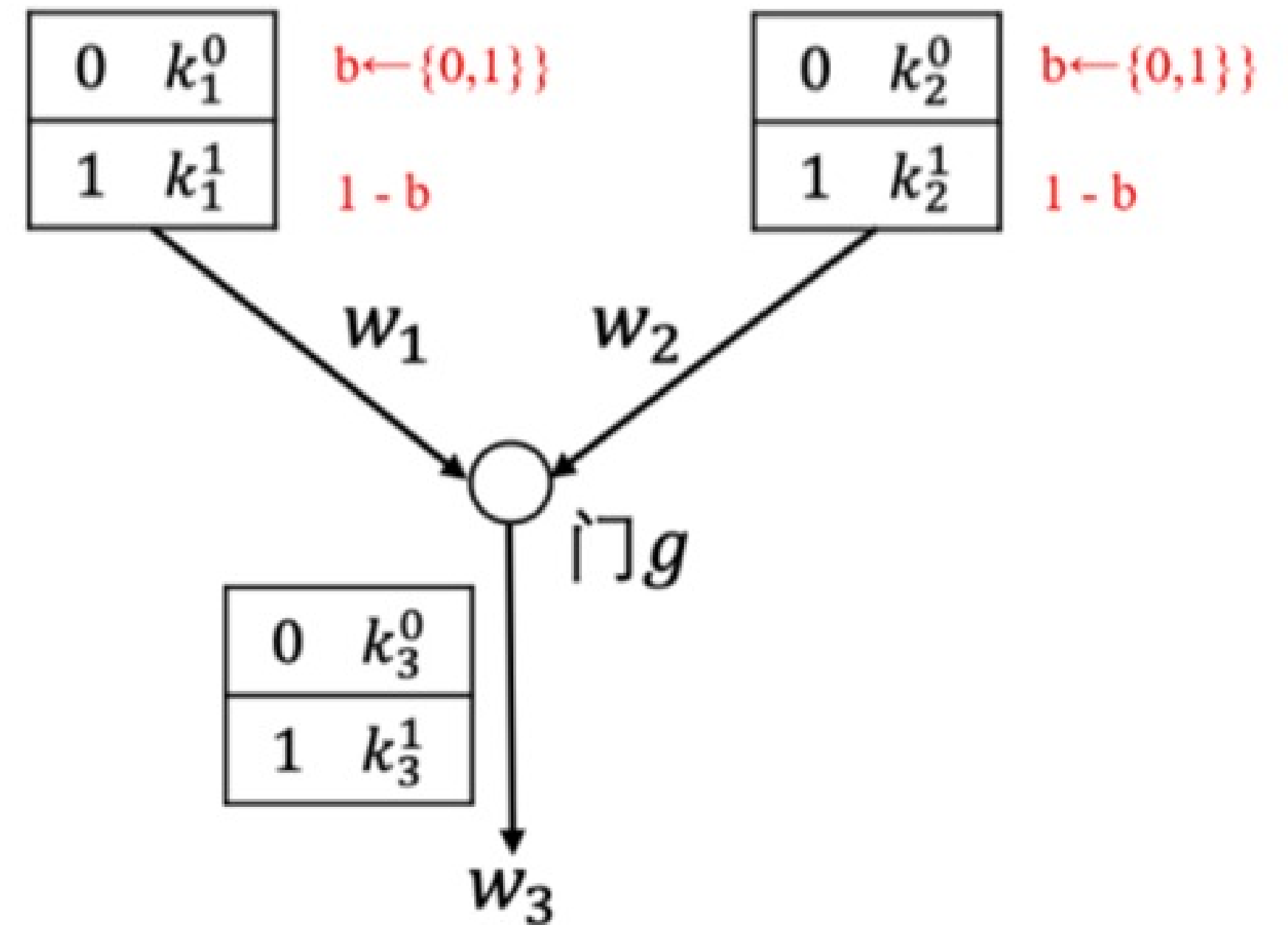
- (1) 只需要2个比特长的置换标识，可以在**为每条导线选择对应的随机密钥的时候**，随机为其产生对应的**1位置换比特**（图中红色部分）即可；
- (2) 任意两个输入(w_1, w_2)的密文存储到两个密钥关联的置换比特所标识的位置。



4. 混淆电路生成

随机置换查找表。 $T_G = \{c_{0,0}, c_{0,1}, c_{1,0}, c_{1,1}\}$ 就是 P_1 构建的**初始查找表**，查找表中的每一行条目都是门输出值所对应导线标签的密文。考虑安全性，需要对查找表进行随机置乱。

比如，如果为 k_1^0 随机选择的置换比特为 1，则 k_1^1 的置换比特则被设置为 0，同样，如果为 k_2^0 随机选择的置换比特为 0，则 k_2^1 的置换比特则被设置为 1，在这个情况下，产生的最后的乱码表就是 $T_G = \{c_{1,0}, c_{1,1}, c_{0,0}, c_{0,1}\}$ 。



4. 混淆电路生成

通常把置换后的查找表称为**乱码表** (Garbled Table) 或**乱码门** (Garbled Gate) 。

电路生成方 P_1 将所有乱码表发送给 P_2 。

所有的乱码门就构成了混淆电路。

5. 混淆电路求值

在收到乱码表后，电路求值方 P_2 开始对电路求值。

- 对于 F 中属于 P_1 的输入导线， P_1 直接将明文相关的密钥（又称为**激活标签**）发给 P_2 。
- 对于 F 中属于 P_2 的输入导线， P_2 通过2选1-OT协议从 P_1 得到对应的激活标签。

激活标签对应的明文值称为**激活值**。

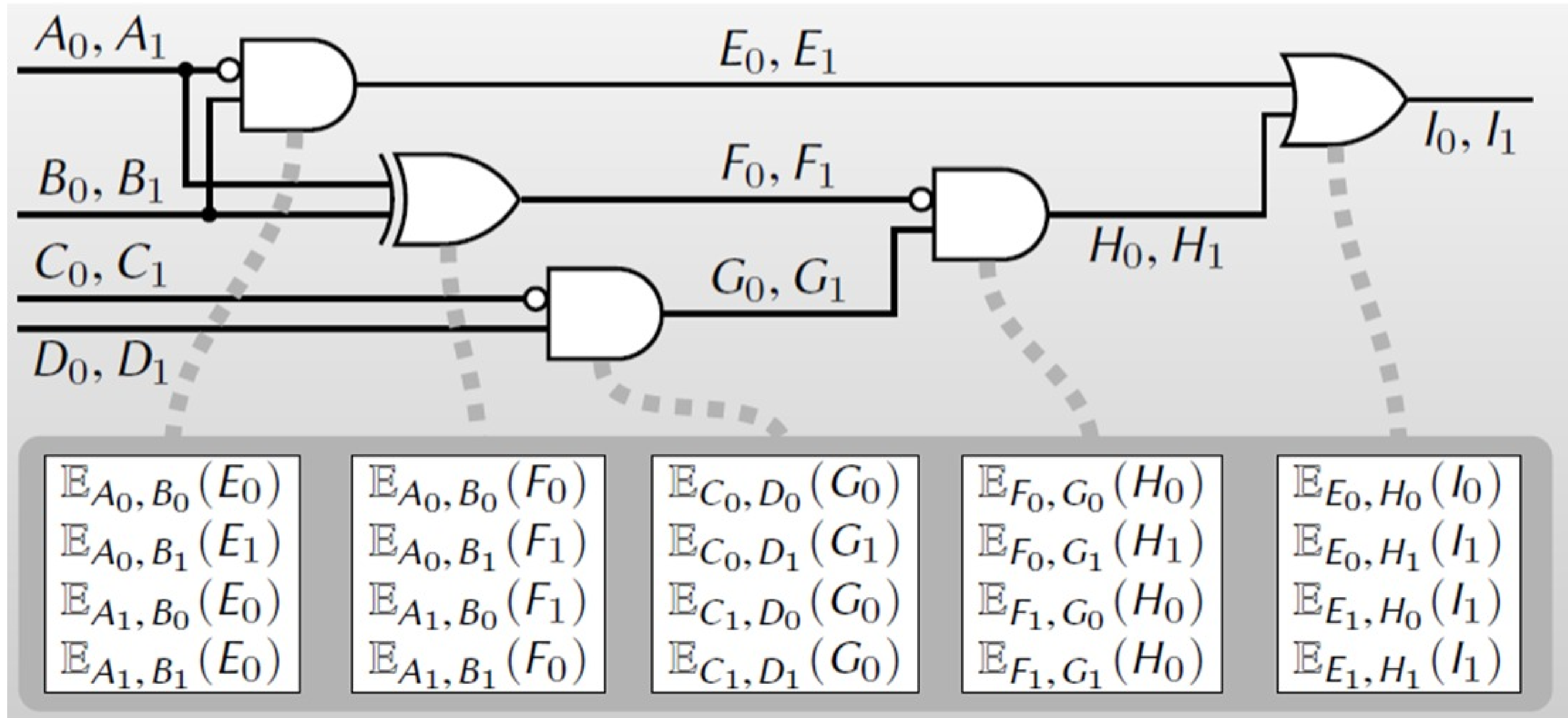
假设门上两条线 w_1, w_2 的输入的值对为 $(0,1)$ ，那么输入线对应的电路计算值为 (k_1^0, k_2^1) 。

k_1^0 是 P_1 直接发送给 P_2 的，而 k_2^1 是通过2选1-OT协议获得的。

5. 混淆电路求值

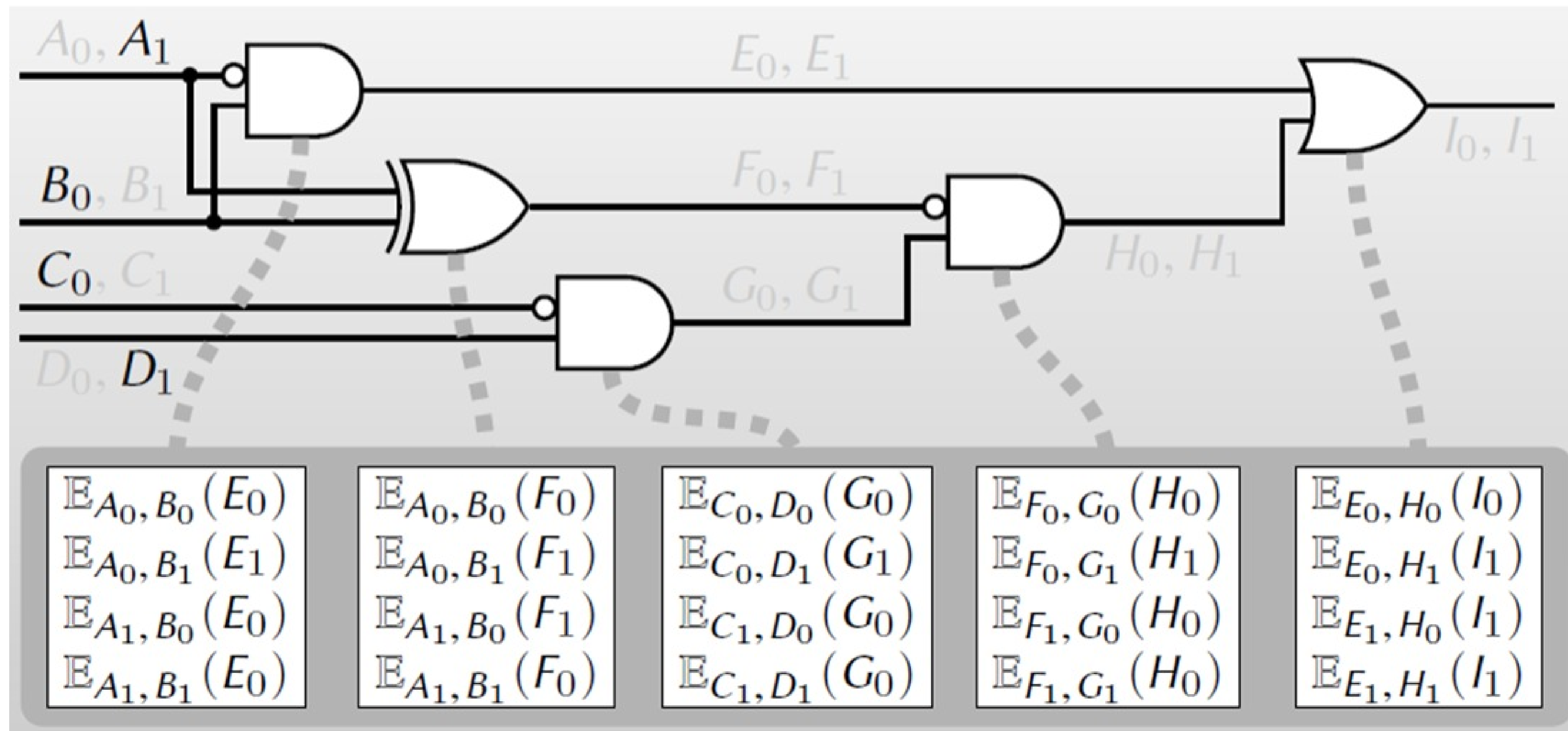
- 因为两个密钥各自蕴含了置换比特，所以，可以**找到对应的要计算的密文 $c_{0,1}$** 。
- 很重要的一点是，对一个乱码门的求解，允许求值方 P_2 得到输出 w_3 对应的激活标签，并在不知道中间激活标签所对应的激活值的条件下，**利用中间激活标签继续做下一个电路门的输入，直到完成 F 的安全求值。**
- 最终， P_2 完成乱码电路的求值，并得到与电路输出导线关联的密钥。 P_2 把得到的密钥发送给 P_1 解密，即可完成 F 的安全求值。

5. 混淆电路求值



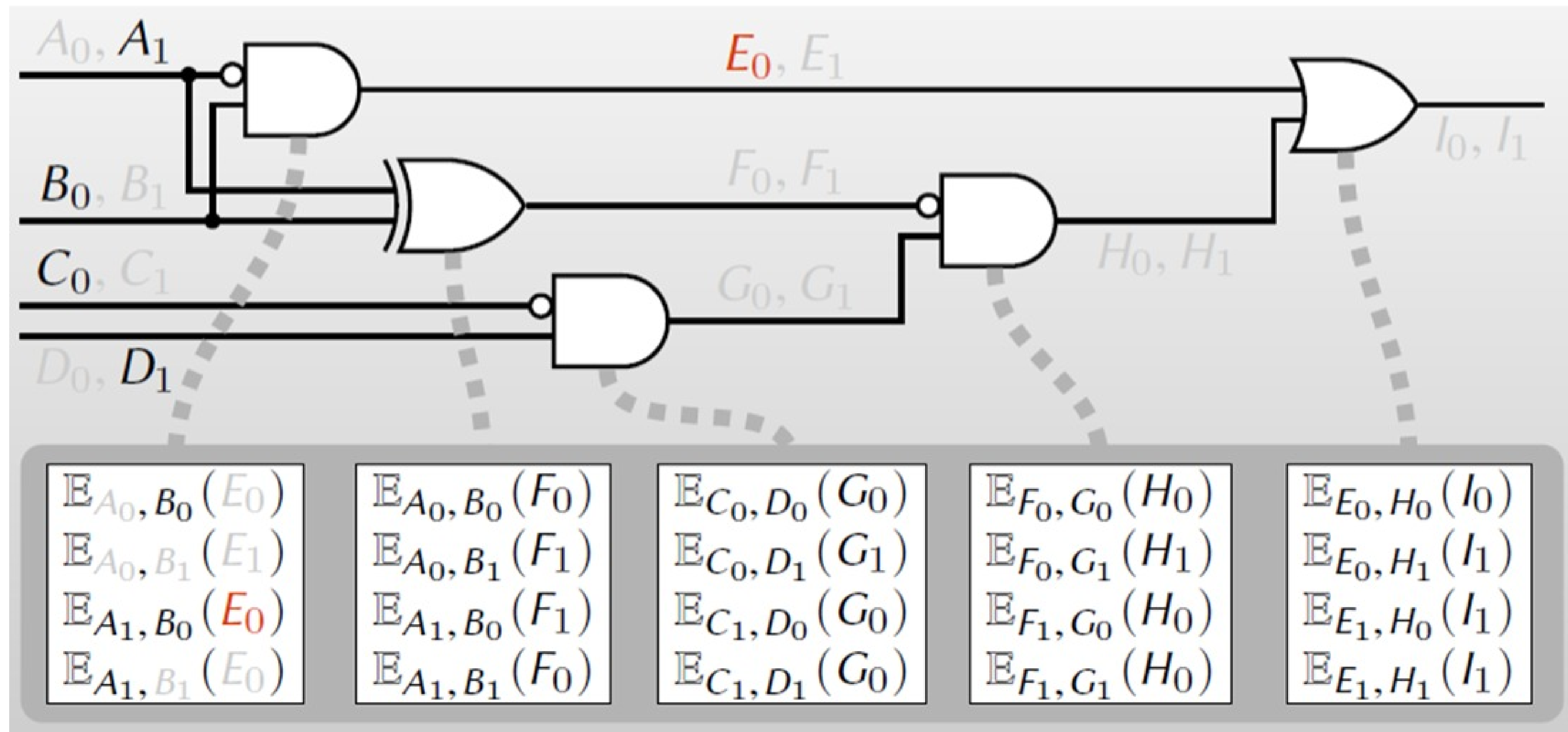
(1) 求值方得到混淆电路

5. 混淆电路求值



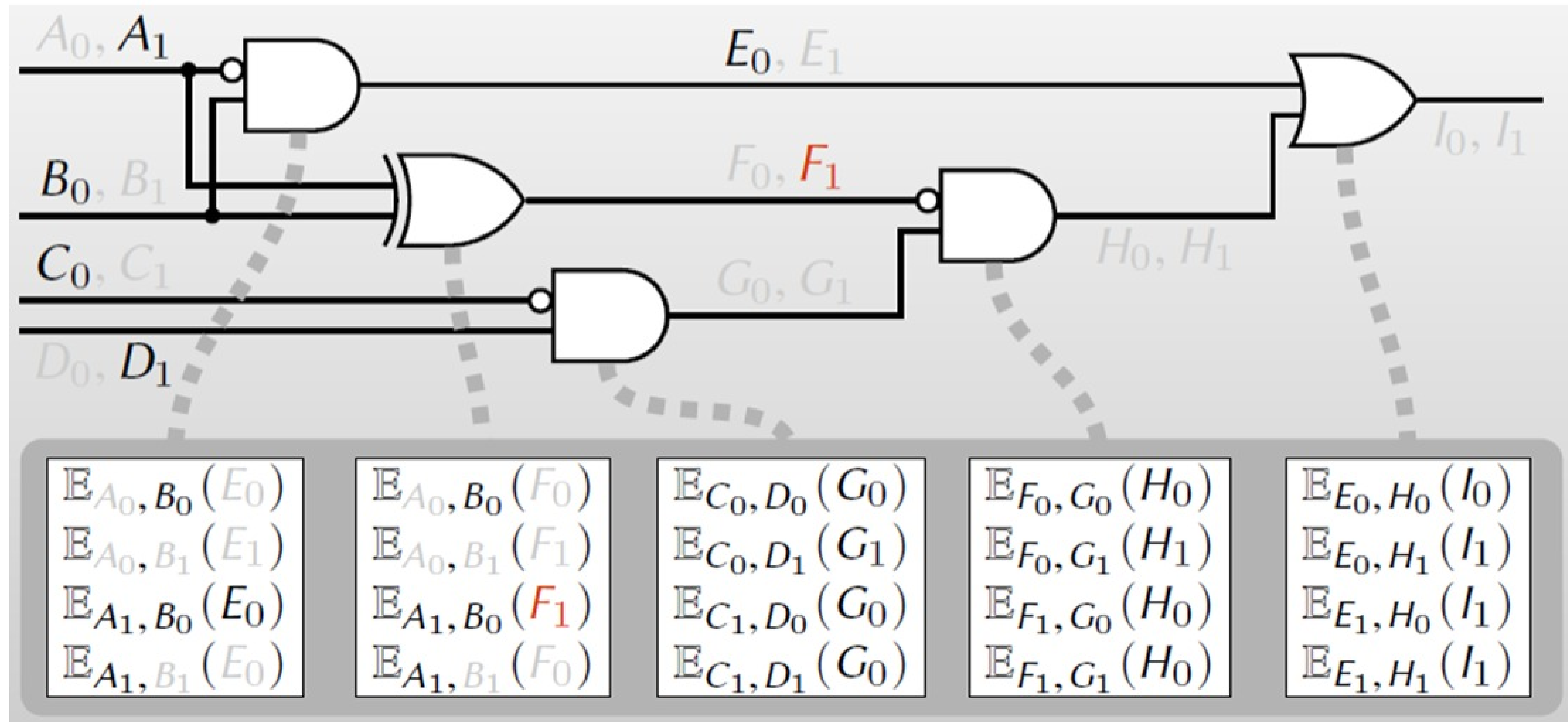
(2) 得到激活标签

5. 混淆电路求值



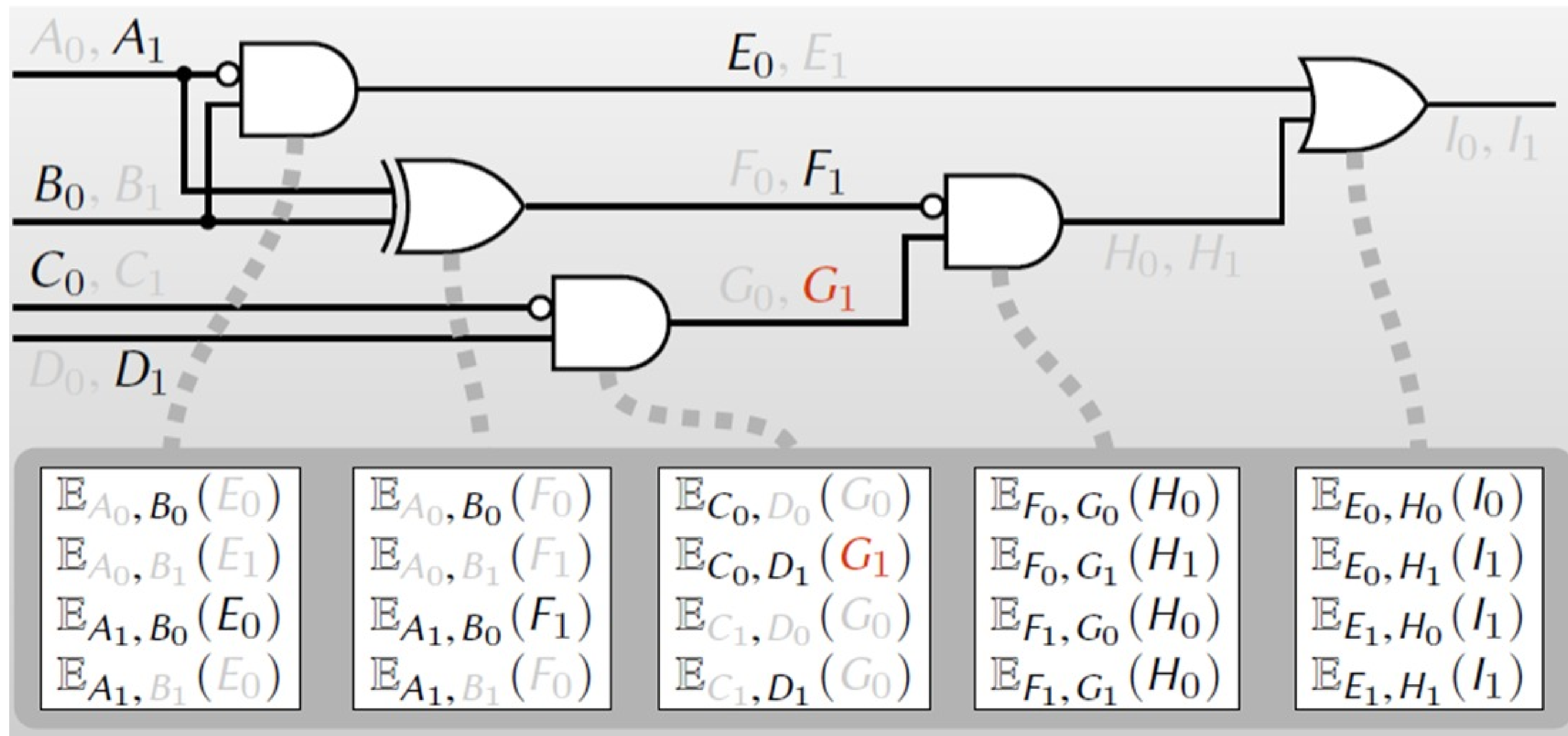
(3) 逐个门开始计算

5. 混淆电路求值



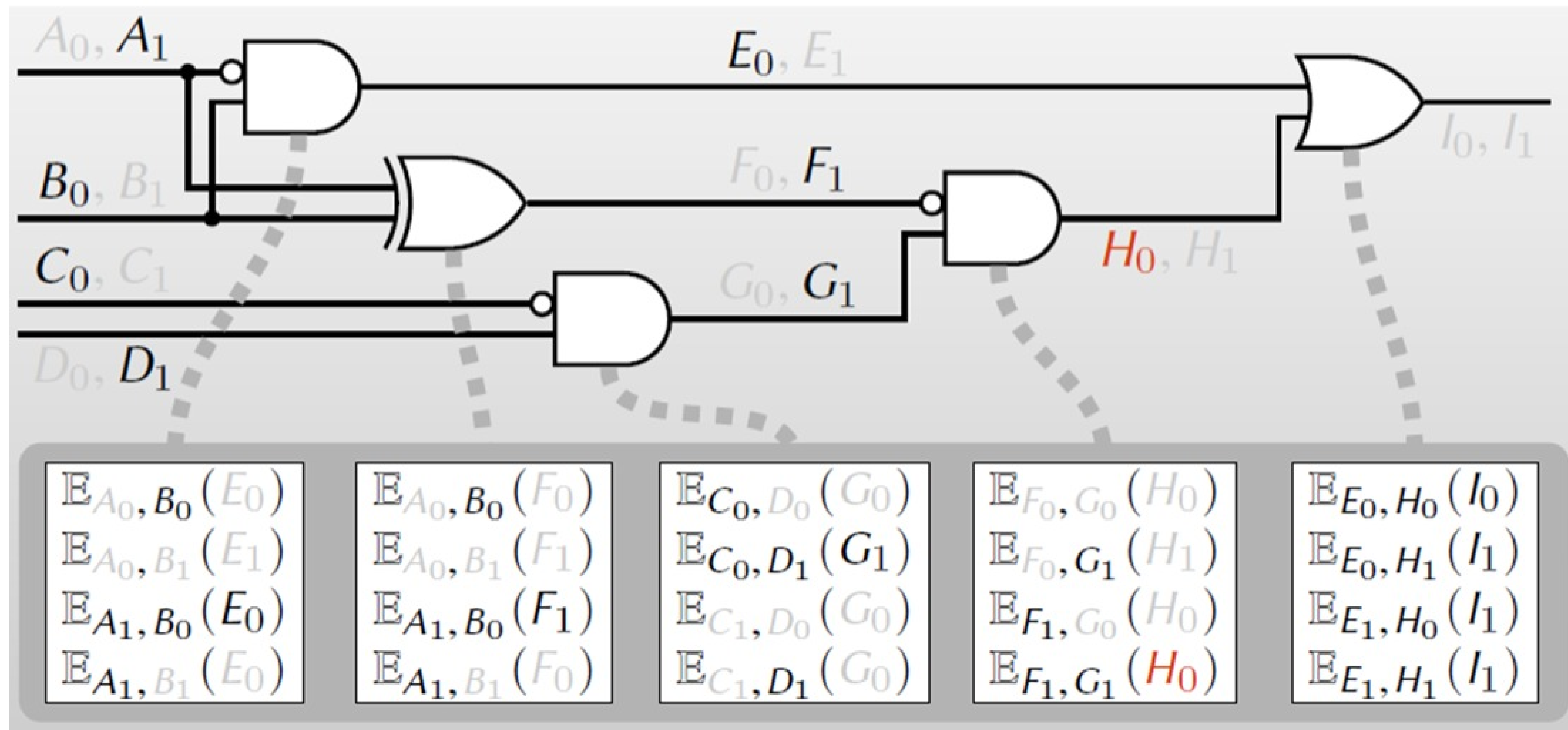
(4) 逐个门开始计算

5. 混淆电路求值



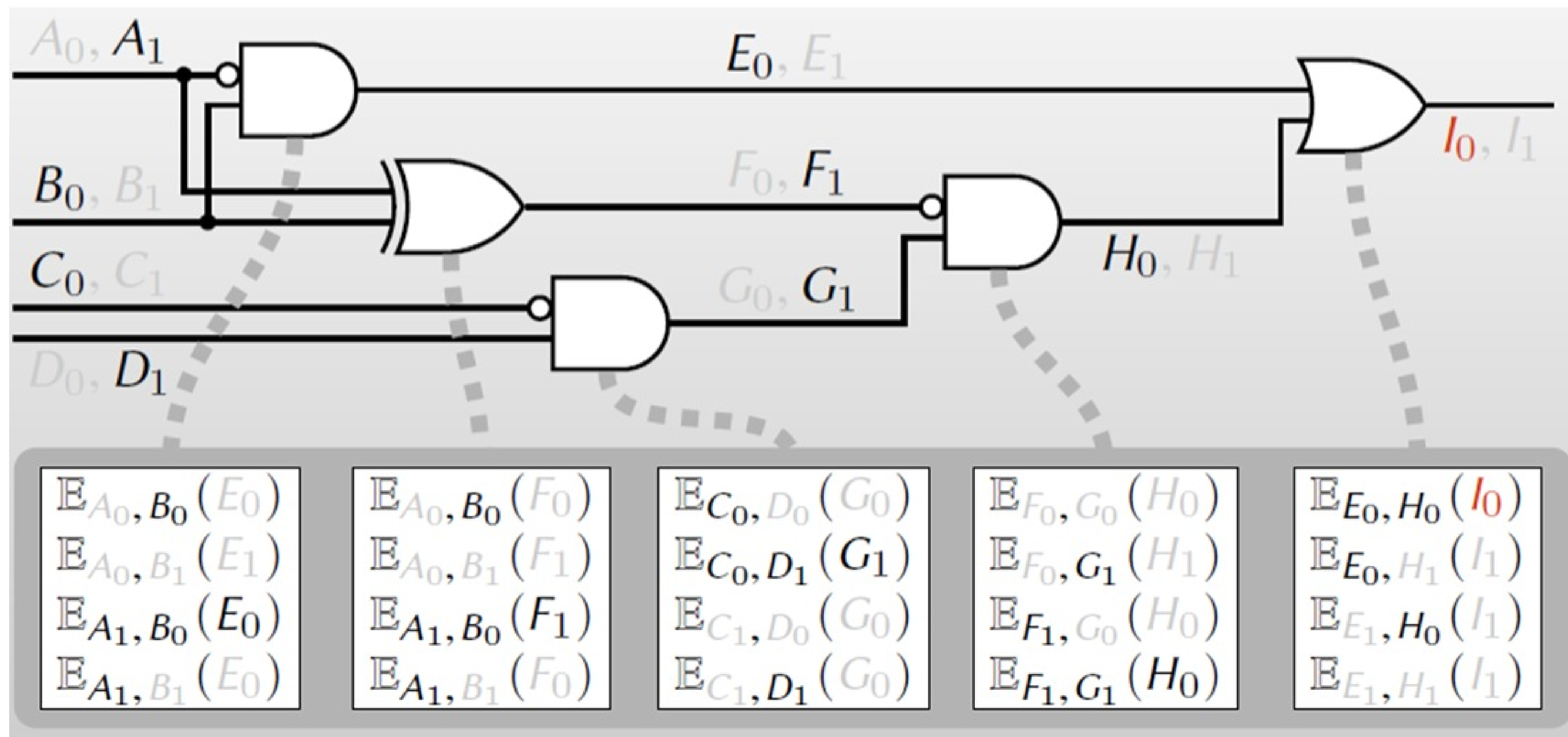
(5) 逐个门开始计算

5. 混淆电路求值



(6) 逐个门开始计算

5. 混淆电路求值



(7) 得到输出的激活标签，双方共同获得对应的激活值

6. 解码表

- 我们注意到， P_2 可以不用将导线标签发送给 P_1 解密，这样可以节省一轮通信过程。
- 具体方法是让 P_1 在发送乱码电路的同时发送输出导线的解码表。
- 解码表只是将输出导线的每个导线标签（随机数）映射为对应的导线值（即相应的明文值），不影响安全性。此时，得到输出导线标签的 P_2 可以在解码表中直接查找导线标签所对应的导线值，得到明文输出。

目 录

『01』

知识点一：姚氏百万富翁问题

『02』

知识点二：混淆电路构造思想

『03』

知识点三：混淆电路求解示例

『04』

知识点四：姚氏乱码电路协议

1. 百万富翁问题的逻辑电路

混淆电路要求要计算的函数能被逻辑电路表示，所以如何将函数转化为一个逻辑电路是关键的一步。以的姚氏百万富翁问题为例，尝试将这一大小比较的函数转化为电路。

- 将Alex和Bob的财富是用二进制表示为 $a_n a_{n-1} \dots a_1, b_n b_{n-1} \dots b_1$ ，其中 $a_i, b_i \in \{0,1\}$ 。对其逐位比较，并可以用归纳法来判断它们的大小。
- 定义变量 c_i 并定义初始值 $c_1 = 0$ ：

$$c_i := \begin{cases} 1 & \text{if } a_{i-1} \dots a_1 > b_{i-1} \dots b_1 \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

- 在已知 a_i, b_i, c_i 的情况下， c_{i+1} 可以做如下推导。

$$c_{i+1} = 1 \Leftrightarrow (a_i > b_i) \text{ or } (a_i = b_i \text{ and } c_i = 1) \quad (2)$$

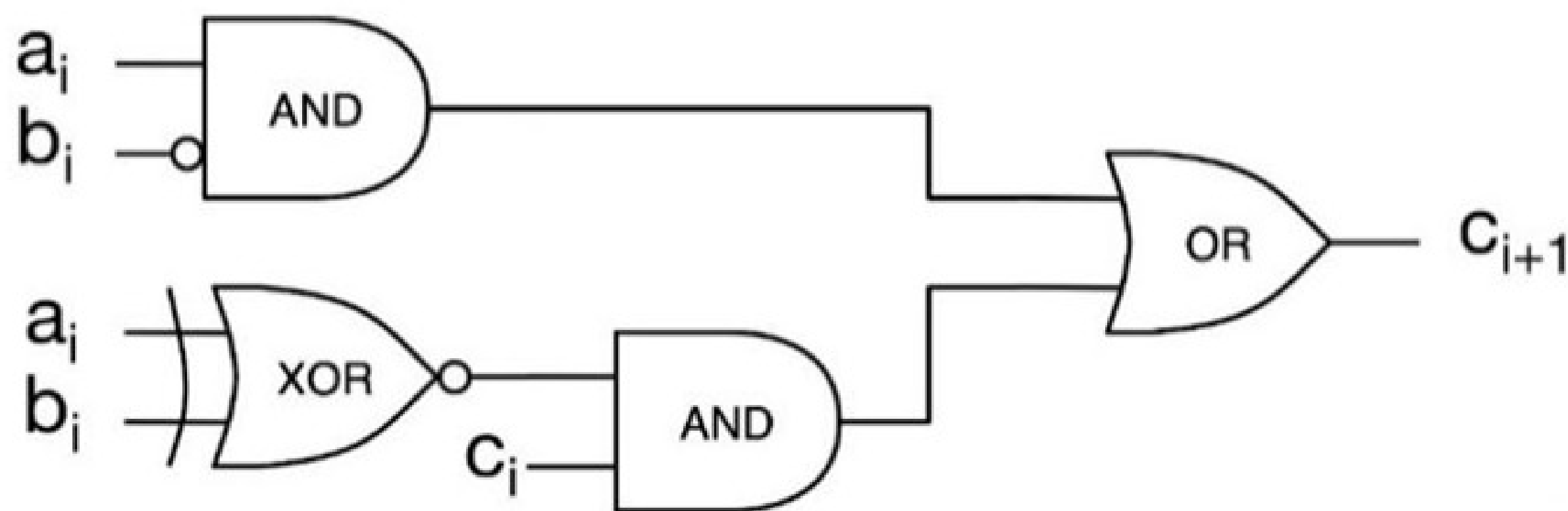
1. 百万富翁问题的逻辑电路

$$c_{i+1} = 1 \Leftrightarrow (a_i > b_i) \text{ or } (a_i = b_i \text{ and } c_i = 1) \quad (2)$$

由于 $a_i, b_i, c_i \in \{0,1\}$, 用 $\sim$ 表示取反,

- $a_i > b_i$ 可以表示为 $a_i \text{ and } \sim b_i$,
- $a_i = b_i$ 可以表示为 $\sim(a_i \text{ xor } b_i)$,

公式 (2) 可以转化成右侧逻辑电路
(图中圆圈表示取反)。

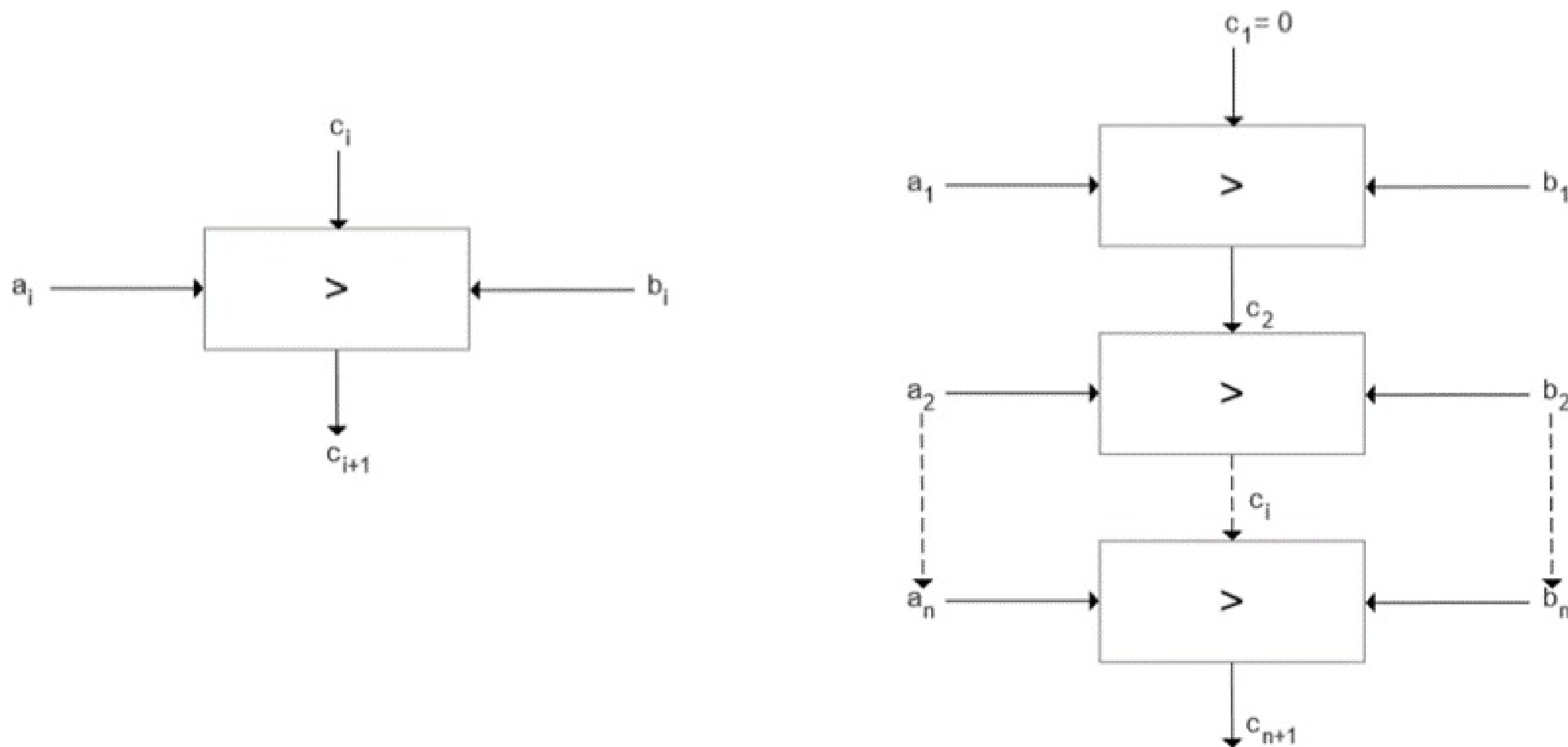


验证: $1 > 0$ $1 \text{ and } \sim 0$

验证: $1 = 1$ $\sim(1 \text{ xor } 1)$

1. 百万富翁问题的逻辑电路

我们将上述电路封装成一个三个输入 (a_i, b_i, c_i) 、一个输出 (c_{i+1}) 的模块 $>$ 。我们将 n 个这样的模块串联起来，就完成了判断 $a_n a_{n-1} \dots a_1 > b_n b_{n-1} \dots b_1$ 的电路。



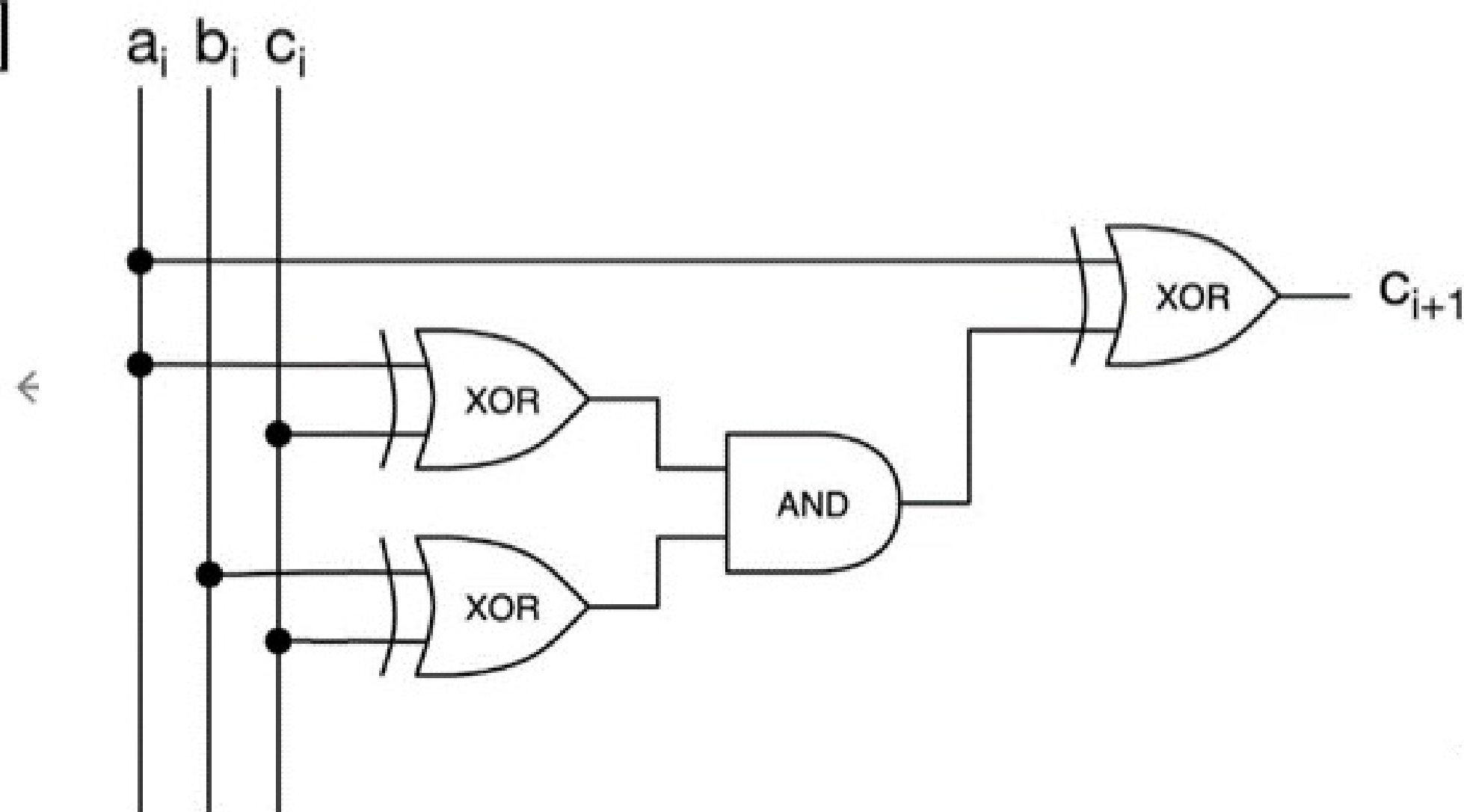
该电路中， c_{n+1} 为整个电路的输出。当输出是1时， $a_n a_{n-1} \dots a_1 > b_n b_{n-1} \dots b_1$ 成立。

1. 百万富翁问题的逻辑电路

上面的电路用到了多处取反，并不是最优的。通过观察，我们不难发现

$c_{i+1} = a_i \oplus [(a_i \oplus c_i) \wedge (b_i \oplus c_i)]$ 。电路可以优化为右侧图。

a_i	b_i	c_i	c_{i+1}	$a_i \oplus [(a_i \oplus c_i) \wedge (b_i \oplus c_i)]$
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	0	1	1
1	1	0	0	0
1	1	1	1	1

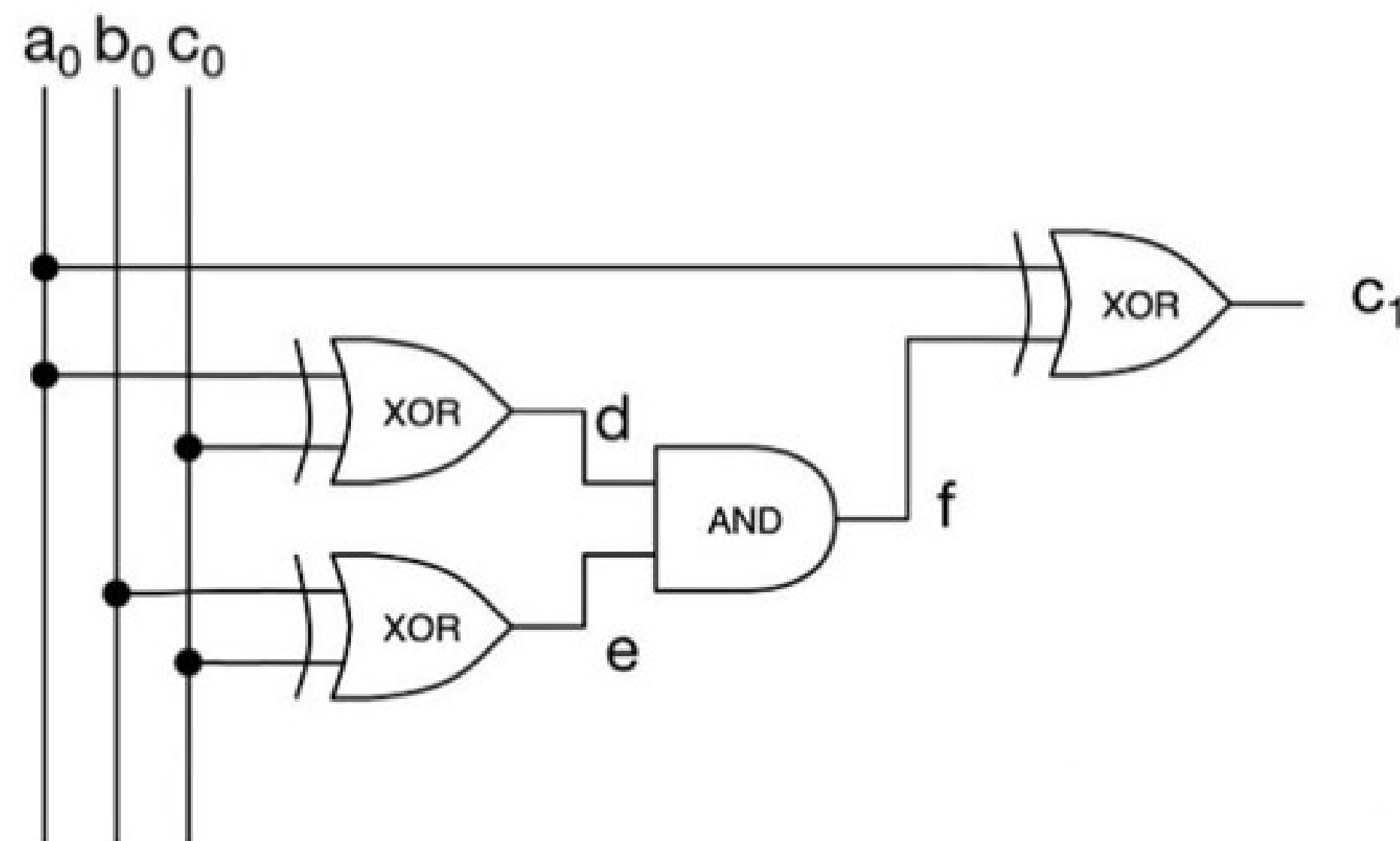


2. 电路生成

Step 1: Alice 生成混淆电路

第一步，Alice对电路中的每一线路（Wire）进行标注。如下图所示，Alice一共标注了七条线路，包括模块的输入输出 $W_{a_0}, W_{b_0}, W_{c_0}, W_{c_1}$ ，和模块内的中间结果 W_d, W_e, W_f 。对于每一条线路 W_i ，Alice生成两个长度为 k 的字符串 X_i^0, X_i^1 。这两个字符串分别对应逻辑上的0和1。

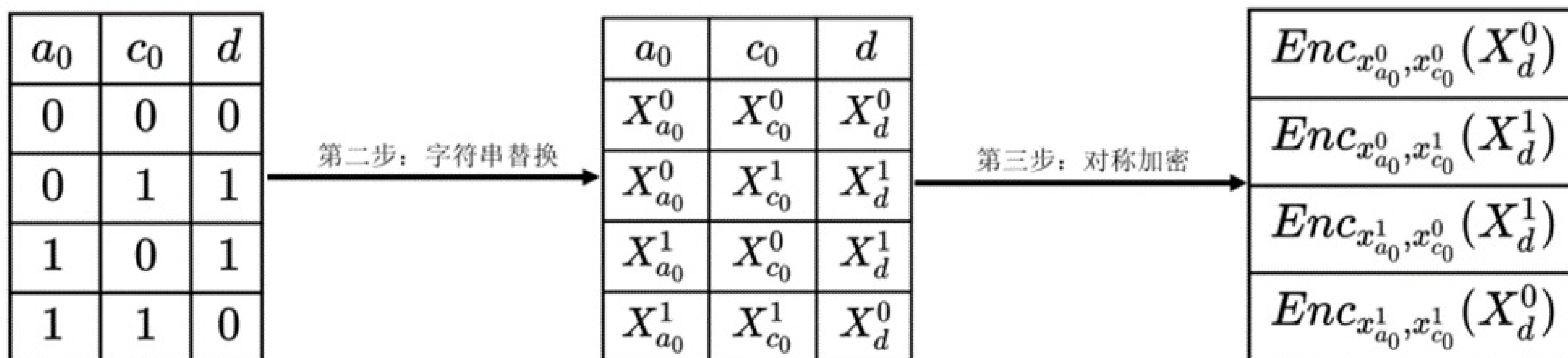
这些生成的标注会在Step 2有选择性地发给Bob，但Bob并不知道 X_i^0, X_i^1 对应的逻辑值。



2. 电路生成

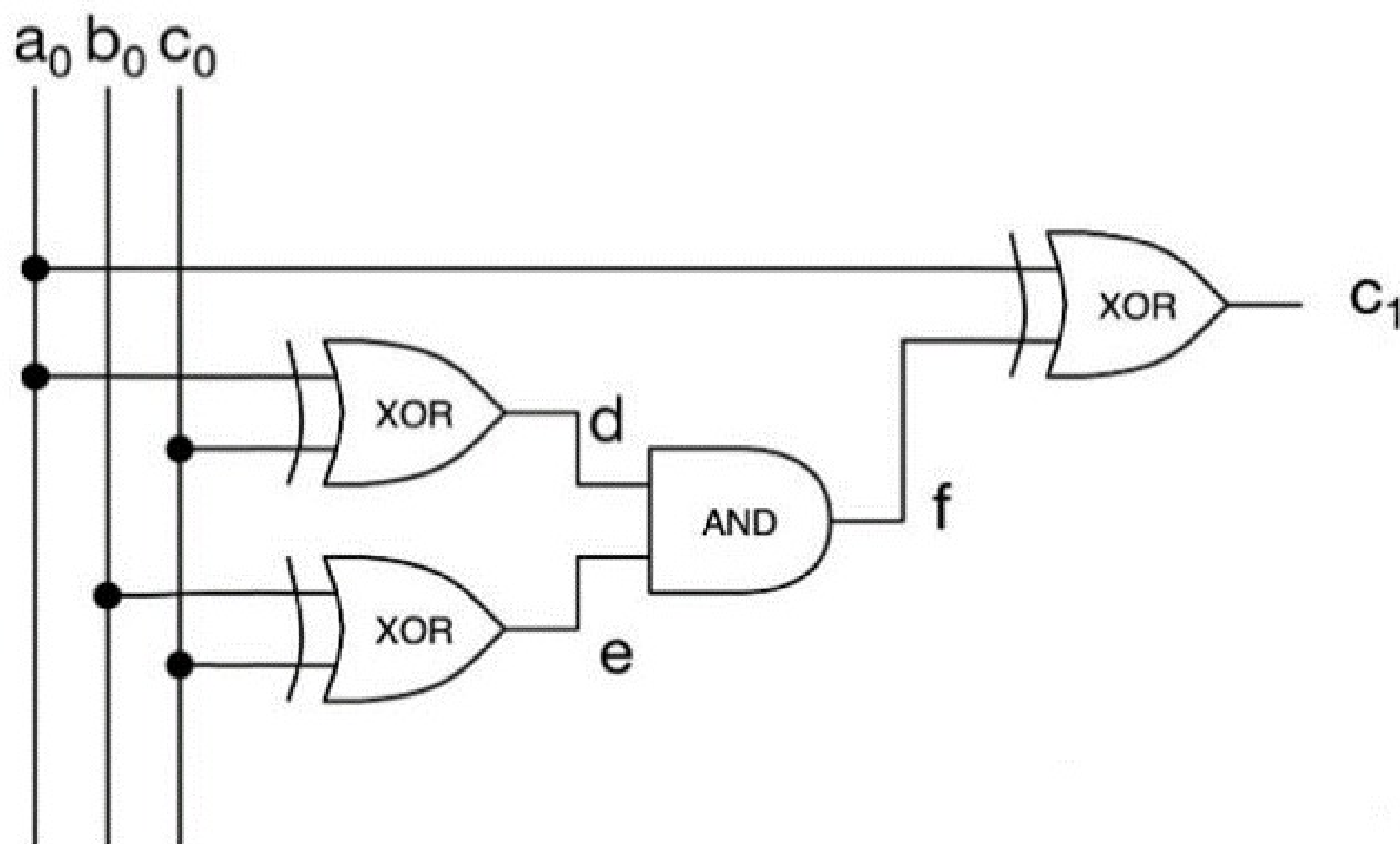
第二步，Alice对电路中的每一个逻辑门的真值表用 X_i^0, X_i^1 进行替换，由 X_i^0 替换 0，由 X_i^1 替换1。以左上方的XOR门的输入是 a_0, c_0 ，输出是 d 为例，见下图变换。

第三步，Alice对每一个替换后的真值表的输出进行两次对称加密，加密的密匙是真值表对应行的两个输入。比如真值表的第一行是 $X_{a_0}^0, X_{c_0}^0, X_d^0$ ，我们就用 $X_{a_0}^0, X_{c_0}^0$ 来加密 X_d^0 ，生成 $Enc_{X_{a_0}^0, X_{c_0}^0}(X_d^0)$ 。



2. 电路生成

对于中间的与门，Alice（电路生成方）将以前面的两个门输出的 d 和 e 的导线标签参与门的生成，即用 X_d^0 、 X_e^1 、 X_d^0 、 X_e^1 参与对称加密。



2. 电路生成

Step 2: Alice和Bob通信

- **第一步，Alice将她的输入对应的字符串发送给Bob。** 比如 $a_0 = 1$ ，那Alice会发送 $X_{a_0}^1$ 给Bob。由于Bob不知道 $X_{a_0}^1$ 对应的逻辑值，也就无从知晓Alice的秘密了。
- **第二步，Bob通过不经意传输协议从Alice获得他的输入对应的字符串。** 不经意传输保证了Bob在 $\{X_{b_0}^0, X_{b_0}^1\}$ 中获得一个，且Alice不知道Bob获得了哪一个。所以Alice也就无从知晓Bob的 b_0 了。
- **最后，Alice 将所有逻辑门的乱码表都发给 Bob。** 本例中共有四个乱码表。

3. 电路求解

Step 3: Bob求解生成的混淆电路

Alice和Bob通信完成之后，Bob便开始沿着电路进行解密。

因为Bob拥有所有输入的标签和所有乱码表，他可以逐一对每个逻辑门的输出进行解密。因为比较大小操作是一个迭代的过程，假设当前轮迭代中，Bob拥有的输入标签为 $X_{a_0}^1, X_{b_0}^1, X_{c_0}^0$ ，他可以：

- 1.对于电路图左上方的 XOR，用 $X_{a_0}^1, X_{c_0}^0$ 解密获得 X_d^1 ；
- 2.对于电路图左下方的 XOR，用 $X_{b_0}^1, X_{c_0}^0$ 解密获得 X_e^1 ；
- 3.对于电路图中间的 AND，用 X_d^1, X_e^1 解密获得 X_f^1 ；
- 4.对于电路图右侧的 XOR，用 $X_{a_0}^1, X_f^1$ 解密 $X_{c_1}^0$ 。

3. 电路求解

Step 4: 共享结果

最后Alice和Bob共享结果，Alice分享 $\{X_{c_1}^0, X_{c_1}^1\}$ 或者Bob分享 $X_{c_1}^1$ ，双方就能获得电路输出的逻辑值了。

目 录

『01』

知识点一：姚氏百万富翁问题

『02』

知识点二：混淆电路构造思想

『03』

知识点三：混淆电路求解示例

『04』

知识点四：姚氏乱码电路协议

参数：

- 实现函数 $\mathcal{F}$ 的布尔电路 $\mathcal{C}$ 。
- 安全参数 κ 。

生成乱码电路：

1. 生成导线标签。对于 $\mathcal{C}$ 的每一条导线 w_i ，随机选择导线标签：
 - (a) $w_i^b = (k_i^b \in_R \{0,1\}^\kappa, p_i^b \in_R \{0,1\})$ 。
 - (b) 使得 $p_i^b = 1 - p_i^{1-b}$ 。
2. 构造乱码电路。对于 $\mathcal{C}$ 的每一个门 G_i ，按照拓扑顺序执行下述步骤：

图 3.1 姚氏乱码电路协议：生成过程

(a) 假设 G_i 是一个实现函数 $g_i: w_c = g_i(w_a, w_b)$ 的 2-输入布尔门, 其中输入导线标签为 $w_a^0 = (k_a^0, p_a^0)$, $w_a^1 = (k_a^1, p_a^1)$, $w_b^0 = (k_b^0, p_b^0)$, $w_b^1 = (k_b^1, p_b^1)$, 输出导线标签为 $w_c^0 = (k_c^0, p_c^0)$, $w_c^1 = (k_c^1, p_c^1)$ 。

(b) 构造 G_i 的乱码表。 G_i 的输入值为 $v_a, v_b \in \{0, 1\}$, 输入值共有 2^2 种可能的组合。对于每一种组合, 设

$$e_{v_a, v_b} = H(k_a^{v_a} \parallel k_b^{v_b} \parallel i) \oplus w_c^{g_i(v_a, v_b)}$$

根据输入导线标签上的置换标识对乱码表中的条目 e 排序, 将条目 e_{v_a, v_b} 放置在位置 $\langle p_a^{v_a}, p_b^{v_b} \rangle$ 上。

乱码表和解码表第二密钥不同

值不同: 导线标签VS真值

3. 输出解码表。对于每一条输出导线 w_i (此导线也是门 G_j 的输出导线), 假设其对应的导线标签为 $w_i^0 = (k_i^0, p_i^0)$, $w_i^1 = (k_i^1, p_i^1)$, 要为两个可能的导线值 $v \in \{0, 1\}$ 创建解码表。设

$$e_v = H(k_i^v \parallel \text{"out"} \parallel j) \oplus v$$

(因为是逐比特执行异或运算, 所以只需要使用 H 输出的最低位生成 e_v)。根据导线标签上的置换标识对解码表中的条目 e 排序, 将条目 e_v 放置在位置 p_i^v 上 (因为 $p_i^1 = p_i^0 \oplus 1$, 所以放置的位置不会发生冲突)。

图 3.1 (续)

参数：

- 参与方 P_1 和 P_2 ，其输入分别为 $x \in \{0,1\}^n$ 和 $y \in \{0,1\}^n$ 。
- 实现函数 $\mathcal{F}$ 的布尔电路 C 。

协议：

1. P_1 的角色为乱码电路生成方，执行图 3.1 所示的步骤。随后， P_1 将得到的乱码电路 $\hat{C}$ （包括输出解码表）发送给 P_2 。
2. 对于 P_1 需要提供输入值的导线， P_1 向 P_2 发送输入值所对应的激活标签。
3. 对于 P_2 需要提供输入值的导线 w_i ， P_1 作为发送方， P_2 作为接收方，两方执行 OT 协议：
 - (a) P_1 的秘密值为导线上的两个导线标签， P_2 的选择比特输入值为 P_2 在此条导线上的输入值。
 - (b) OT 协议执行完毕后， P_2 得到导线的激活标签。
4. P_2 从输入导线的激活标签开始，按照拓扑顺序逐门对收到的 $\hat{C}$ 求值。
 - (a) 对于乱码表为 $T=(e_{0,0}, \dots, e_{1,1})$ ，输入激活标签为 $w_a=(k_a, p_a)$ 和 $w_b=(k_b, p_b)$ 的门 G_i ， P_2 计算输出激活标签 $w_c=(k_c, p_c)$ 。

$w_c = H(k_a \parallel k_b \parallel i) \oplus e_{p_a, p_b}$

得到标签
5. 应用输出解码表得到输出。当对 $\hat{C}$ 的所有门完成求值后， P_2 把第二个密钥设置为“out”，解码最终的输出门，得到明文计算结果。 P_2 将得到的计算结果发送给 P_1 ，双方均将计算结果作为协议的输出。

图 3.2 姚氏乱码电路协议：求值过程

谢 谢

